



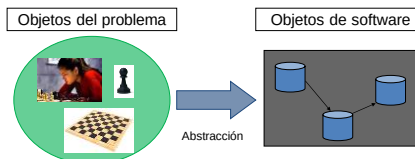
Depto. de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur, Bahía Blanca

En esta clase

- Clases proveedoras y clientes
- Variables, objetos y referencias
- Diagrama de objetos
- Métodos y mensajes
- Identidad y estado interno
- Parámetros y resultados
- Ejemplo: Cajero automático

01100
10011
10110
01110
01100
10011
10110
1001
1 11
0 0
1

El modelo computacional

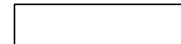


El **modelo computacional** de la POO es un mundo poblado de **objetos** comunicándose a través de **mensajes**. Cada objeto responde a un mensaje de acuerdo al comportamiento determinado por su clase.

01100
10011
10110
01110
01100
10011
10110
1001
1 11
0 0
1

El diagrama de una clase

Nombre
<<Atributos de clase>> <<Atributos de instancia>>
<<Constructores>> <<Comandos>> <<Consultas>>
Responsabilidades



01100
10011
10110
01110
01100
10011
10110
1001
1 11
0 0
1

Los servicios se clasifican en **constructores**, **comandos** y **consultas**. Los comandos y consultas son los **métodos** provistos por la clase.

Constructores

Un **constructor** es un **servicio** provisto por la clase y se caracteriza porque recibe el **mismo nombre que la clase**.

El constructor **se invoca cuando se crea un objeto** y habitualmente se usa para **inicializar** los valores de los atributos de instancia.

Una clase puede brindar **varios constructores**, siempre que tengan diferente número o tipo de parámetros.

Si en una clase no se define explícitamente un constructor, el compilador crea automáticamente uno.

Si la clase incluye uno o más constructores, el compilador no agrega ningún otro implícitamente.

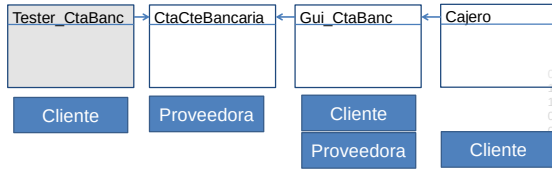
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1

Comandos y consultas

- Los **comandos** son servicios que modifican los valores de los atributos de un objeto.
- Las **consultas** son servicios que no modifican los valores de los atributos, en general, devuelven un resultado que corresponde al valor de un atributo o al cómputo de una expresión.
- Un comando puede retornar también un valor.
- Los comandos y consultas conforman los **métodos** de una clase.
- Con frecuencia una clase incluye un comando para modificar cada atributo de instancia y una consulta para retornar el valor de cada atributo de instancia. Los llamamos **comandos y consultas triviales**.

01100
10011
10110
01110
01100
10011
10110
1001
1 11
0 0
1

Ejemplo: Cajero automático



El diagrama de clases establece las **responsabilidades** de cada una de las clases.

Ejemplo: Cajero automático

CtaCteBancaria	CtaCteBancaria(c:entero) Requiere c > 0
<<Atributo de clase>> maxDescubierto = 1000	CtaCteBancaria(c:entero,s:real) Requiere c > 0 y s >= 0
<<Atributos de instancia>> codigo:entero saldo:real	depositar(mto:real) Requiere mto > 0 Incrementa el saldo en mto
<<Constructores>> CtaCteBancaria(c:entero) CtaCteBancaria(c:entero,s:real)	extraer(mto:real):boolean Requiere mto > 0 Si mto > saldo+maxDescubierto retorna false y la extracción no se realiza, sino decrementa el saldo en mto y retorna true
<<Comandos>> depositar(mto:real) extraer(mto:real):boolean	
<<Consultas>> obtenerCodigo():entero obtenerSaldo():real toString():String mayorSaldo(CtaCteBancaria cta):entero ctaMayorSaldo(CtaCteBancaria cta):CtaCteBancaria	

Ejemplo: Cajero automático

CtaCteBancaria	mayorSaldo(cta:CtaCteBancaria):entero Requiere cta ligada. Si el saldo de la cuenta que recibe el mensaje es mayor que el saldo de cta, retorna el código de la cuenta que recibe el mensaje, sino retorna el código de cta.
<<Atributo de clase>> maxDescubierto = 1000	ctaMayorSaldo(cta:CtaCteBancaria):CtaCteBancaria Requiere cta ligada. Si el saldo de la cuenta que recibe el mensaje es mayor que el saldo de cta, retorna la cuenta que recibe el mensaje, sino retorna cta.
<<Atributos de instancia>> codigo:entero saldo:real	
<<Constructores>> CtaCteBancaria(c:entero) CtaCteBancaria(c:entero,s:real)	
<<Comandos>> depositar(mto:real) extraer(mto:real):boolean	
<<Consultas>> obtenerCodigo():entero obtenerSaldo():real toString():String mayorSaldo(CtaCteBancaria cta):entero ctaMayorSaldo(CtaCteBancaria cta):CtaCteBancaria	

Ejemplo: Cajero automático

CtaCteBancaria	class CtaCteBancaria{ //Asegura código > 0 y saldo >= - maxDescubierto //Atributos de clases private static final int maxDescubierto=1000;
<<Atributo de clase>> maxDescubierto = 1000	
<<Atributos de instancia>> codigo:entero saldo:real	
<<Constructores>> CtaCteBancaria(c:entero) CtaCteBancaria(c:entero,s:real)	
<<Comandos>> depositar(mto:real) extraer(mto:real):boolean	
<<Consultas>> obtenerCodigo():entero obtenerSaldo():real toString():String mayorSaldo(CtaCteBancaria cta):entero ctaMayorSaldo(CtaCteBancaria cta):CtaCteBancaria	

La variable **maxDescubierto** es un atributo de clase, todos los objetos de la clase comparten un mismo valor.

Ejemplo: Cajero automático

CtaCteBancaria	class CtaCteBancaria{ //Asegura código > 0 y saldo >= - maxDescubierto //Atributos de clases private static final int maxDescubierto=1000; //Atributos de Instancia /*El código se establece al crear la cuenta y no cambia*/ private int codigo; private float saldo;
<<Atributo de clase>> maxDescubierto = 1000	
<<Atributos de instancia>> codigo:entero saldo:real	
<<Constructores>> CtaCteBancaria(c:entero) CtaCteBancaria(c:entero,s:real)	
<<Comandos>> depositar(mto:real) extraer(mto:real):boolean	
<<Consultas>> obtenerCodigo():entero obtenerSaldo():real toString():String mayorSaldo(CtaCteBancaria cta):entero ctaMayorSaldo(CtaCteBancaria cta):CtaCteBancaria	

Las variables **codigo** y **saldo** son los atributos de instancia de la clase y pueden ser usados en cualquiera de los servicios provistos por la clase **CtaCteBancaria**.

Ejemplo: Cajero automático

CtaCteBancaria	class CtaCteBancaria{ //Asegura código > 0 y saldo >= - maxDescubierto //Atributos de clases private static final int maxDescubierto=1000; //Atributos de Instancia /*El código se establece al crear la cuenta y no cambia*/ private int codigo; private float saldo;
<<Atributo de clase>> maxDescubierto = 1000	
<<Atributos de instancia>> codigo:entero saldo:real	
<<Constructores>> CtaCteBancaria(c:entero) CtaCteBancaria(c:entero,s:real)	
<<Comandos>> depositar(mto:real) extraer(mto:real):boolean	
<<Consultas>> obtenerCodigo():entero obtenerSaldo():real toString():String mayorSaldo(CtaCteBancaria cta):entero ctaMayorSaldo(CtaCteBancaria cta):CtaCteBancaria	

Como todos los atributos se declaran **privados**, sus valores sólo pueden ser accedidos desde el exterior por los servicios públicos que brinda la clase.

Ejemplo: Cajero automático

CtaCteBancaria	
<<Atributo de clase>> maxDescubierto = 1000	class CtaCteBancaria{ //Asegura código > 0 y saldo >= - maxDescubierto //Atributos de clases private static final int maxDescubierto=1000; //Atributos de Instancia /*El código se establece al crear la cuenta y no cambia*/ private int codigo; private float saldo; // Constructores public CtaCteBancaria(int c) { // Requiere c > 0 codigo = c; saldo = 0; } public CtaCteBancaria(int c, float s) { // Requiere c > 0 y s >= 0 codigo = c; saldo = s; } }
<<Atributos de instancia>> codigo:entero saldo:real	
<<Constructores>> CtaCteBancaria(c:entero) CtaCteBancaria(c:entero,s:real)	
<<Comandos>> depositar(mto:real) extraer(mto:real):boolean	
<<Consultas>> obtenerCodigo():entero obtenerSaldo():real toString():String mayorSaldo(CtaCteBancaria cta):entero ctaMayorSaldo(CtaCteBancaria cta): CtaCteBancaria	

Si siguiendo el diseño, la clase brinda dos constructores.

Ejemplo: Cajero automático

CtaCteBancaria	
<<Atributo de clase>> maxDescubierto = 1000	//Comandos public void depositar(float mto) { /*Incrementa el saldo en mto, Requiere mto > 0*/ saldo += mto; }
<<Atributos de instancia>> codigo:entero saldo:real	
<<Constructores>> CtaCteBancaria(c:entero) CtaCteBancaria(c:entero,s:real)	
<<Comandos>> depositar(mto:real) extraer(mto:real):boolean	
<<Consultas>> obtenerCodigo():entero obtenerSaldo():real toString():String mayorSaldo(CtaCteBancaria cta):entero ctaMayorSaldo(CtaCteBancaria cta): CtaCteBancaria	

El diseñador especificó que:

- el comando **depositar** modifica el valor del atributo de instancia **saldo**.
- es responsabilidad de la clase que usa a **CtaCteBancaria** asegurar que **mto > 0**

Ejemplo: Cajero automático

CtaCteBancaria	
<<Atributo de clase>> maxDescubierto = 1000	//Comandos public void depositar(float mto) { /*Incrementa el saldo en mto, Requiere mto > 0*/ saldo += mto; }
<<Atributos de instancia>> codigo:entero saldo:real	public boolean extraer(float mto) { /*Requiere mto > 0. Si mto > saldo+maxDescubierto retorna false y la extracción no se realiza, en caso contrario decrementa el saldo en mto y retorna true*/ boolean puede = true; if (saldo + maxDescubierto >= mto) saldo=saldo-mto; else puede = false; return puede; }
<<Constructores>> CtaCteBancaria(c:entero) CtaCteBancaria(c:entero,s:real)	
<<Comandos>> depositar(mto:real) extraer(mto:real):boolean	
<<Consultas>> obtenerCodigo():entero obtenerSaldo():real toString():String mayorSaldo(CtaCteBancaria cta):entero ctaMayorSaldo(CtaCteBancaria cta): CtaCteBancaria	

En el comando **extraer** la variable local **puede** se crea cuando se inicia la ejecución del método y solo puede ser accedida en ese bloque de código.

Ejemplo: Cajero automático

CtaCteBancaria	
<<Atributo de clase>> maxDescubierto = 1000	//Comandos public void depositar(float mto) { /*Incrementa el saldo en mto, Requiere mto > 0*/ saldo += mto; }
<<Atributos de instancia>> codigo:entero saldo:real	public boolean extraer(float mto) { /*Requiere mto > 0. Si mto > saldo+maxDescubierto retorna false y la extracción no se realiza, en caso contrario decrementa el saldo en mto y retorna true*/ boolean puede = true; if (saldo + maxDescubierto >= mto) saldo=saldo-mto; else puede = false; return puede; }
<<Constructores>> CtaCteBancaria(c:entero) CtaCteBancaria(c:entero,s:real)	
<<Comandos>> depositar(mto:real) extraer(mto:real):boolean	
<<Consultas>> obtenerCodigo():entero obtenerSaldo():real toString():String mayorSaldo(CtaCteBancaria cta):entero ctaMayorSaldo(CtaCteBancaria cta): CtaCteBancaria	

La variable **mto** es un **parámetro formal**. Se crea cuando se inicia la ejecución del método y se inicializa con el valor del **parámetro real**.

Ejemplo: Cajero automático

CtaCteBancaria	
<<Atributo de clase>> maxDescubierto = 1000	//Comandos public void depositar(float mto) { /*Incrementa el saldo en mto, Requiere mto > 0*/ saldo += mto; }
<<Atributos de instancia>> codigo:entero saldo:real	public boolean extraer(float mto) { /*Requiere mto > 0. Si mto > saldo+maxDescubierto retorna false y la extracción no se realiza, en caso contrario decrementa el saldo en mto y retorna true*/ if (saldo + maxDescubierto >= mto) saldo=saldo-mto; else puede = false; return puede; }
<<Constructores>> CtaCteBancaria(c:entero) CtaCteBancaria(c:entero,s:real)	
<<Comandos>> depositar(mto:real) extraer(mto:real):boolean	
<<Consultas>> obtenerCodigo():entero obtenerSaldo():real toString():String mayorSaldo(CtaCteBancaria cta):entero ctaMayorSaldo(CtaCteBancaria cta): CtaCteBancaria	

Al terminar la ejecución de **extraer** las variables **puede** y **mto** se destruyen.

Ejemplo: Cajero automático

CtaCteBancaria	
<<Atributo de clase>> maxDescubierto = 1000	// Consultas public int obtenerCodigo() { return codigo; }
<<Atributos de instancia>> codigo:entero saldo:real	public float obtenerSaldo() { return saldo; }
<<Constructores>> CtaCteBancaria(c:entero) CtaCteBancaria(c:entero,s:real)	
<<Comandos>> depositar(mto:real) extraer(mto:real):boolean	
<<Consultas>> obtenerCodigo():entero obtenerSaldo():real toString():String mayorSaldo(CtaCteBancaria cta):entero ctaMayorSaldo(CtaCteBancaria cta): CtaCteBancaria	

En cada consulta el tipo que precede al nombre del método es compatible con el tipo del resultado que retorna.

Ejemplo: Cajero automático

CtaCteBancaria
<<Atributo de clase>> maxDescubierto = 1000
<<Atributos de instancia>> codigo:entero saldo:real
<<Constructores>> CtaCteBancaria(c:entero) CtaCteBancaria(c:entero,s:real)
<<Comandos>> depositar(mto:real) extraer(mto:real):boolean
<<Consultas>> obtenerCodigo():entero obtenerSaldo():real toString():String mayorSaldo(CtaCteBancaria cta):entero ctaMayorSaldo(CtaCteBancaria cta): CtaCteBancaria

La clase **CtaCteBancaria** no lee ni muestra datos, toda la entrada y salida la hacen las clases que usan a **CtaCteBancaria**.

Ejemplo: Cajero automático

```
class TestUnaCuenta {
    public static void main (String a[]){

        CtaCteBancaria cb = new CtaCteBancaria (111,250);
        cb.depositar(1000);
        System.out.println(cb.obtenerCodigo()+" "+cb.obtenerSaldo());
    }
}
```

En Java el método **main** que inicia la ejecución

Variables, objetos y referencias

```
class TestUnaCuenta {
    public static void main (String a[]){

        CtaCteBancaria cb = new CtaCteBancaria (111,250);
        cb.depositar(1000);
        System.out.println(cb.obtenerCodigo()+" "+cb.obtenerSaldo());
    }
}
```

cb

- Declara la variable cb.
- Crea un objeto de clase **CtaCteBancaria**.
- Liga el objeto a la variable.

Variables, objetos y referencias

```
class TestUnaCuenta {
    public static void main (String a[]){

        CtaCteBancaria cb = new CtaCteBancaria (111,250);
        cb.depositar(1000);
        System.out.println(cb.obtenerCodigo()+" "+cb.obtenerSaldo());
    }
}
```

cb

```
.CtaCteBancaria
codigo=
saldo=
```

- Crea un objeto de clase **CtaCteBancaria**
- Reserva espacio en memoria para almacenar el estado interno del objeto.
- Ejecuta el bloque del constructor con dos parámetros

Variables, objetos y referencias

```
class TestUnaCuenta {
    public static void main (String a[]){

        CtaCteBancaria cb = new CtaCteBancaria (111,250);
        cb.depositar(1000);
        System.out.println(cb.obtenerCodigo()+" "+cb.obtenerSaldo());
    }
}
```

```
public CtaCteBancaria(int c, float s)
{ // Requiere c >0 y s >= 0
    codigo = c; saldo = s; }
c s
```

cb

```
.CtaCteBancaria
codigo=111
saldo=250
```

Antes de iniciarse la ejecución del bloque se reserva espacio en memoria para mantener los valores de los parámetros formales, que se inicializan con los valores de los parámetros reales.

Variables, objetos y referencias

```
class TestUnaCuenta {
    public static void main (String a[]){

        CtaCteBancaria cb = new CtaCteBancaria (111,250);
        cb.depositar(1000);
        System.out.println(cb.obtenerCodigo()+" "+cb.obtenerSaldo());
    }
}
```

```
public CtaCteBancaria(int c, float s)
{ // Requiere c >0 y s >= 0
    codigo = c; saldo = s; }
cb
```

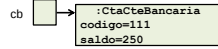
cb

```
.CtaCteBancaria
codigo=111
saldo=250
```

En este ejemplo cada atributo de instancia se inicializa con el valor de un parámetro formal y la ejecución del constructor termina.

Variables, objetos y referencias

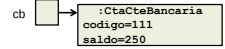
```
class TestUnaCuenta {
    public static void main (String a[]){
        CtaCteBancaria cb = new CtaCteBancaria (111,250);
        cb.depositar(1000);
        System.out.println(cb.obtenerCodigo()+" "+cb.obtenerSaldo());
    }
}
```



Cuando el control vuelve al método **main**, la asignación liga el objeto a la variable, es decir, almacena la **referencia** al objeto en memoria.

Variables, objetos y referencias

El valor de la **variable** **cb** es una **referencia** a un **objeto** de clase **CtaCteBancaria**.

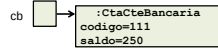


El diagrama de un objeto

```
class TestUnaCuenta {
    public static void main (String a[]){
        CtaCteBancaria cb = new CtaCteBancaria (111,250);
        cb.depositar(1000);
        System.out.println(cb.obtenerCodigo()+" "+cb.obtenerSaldo());
    }
}
```

```
public CtaCteBancaria(int c, float s){
    // Requiere c > 0 y s >= 0
    codigo = c; saldo = s; }

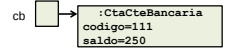
```



El diagrama de objetos es una representación visual para modelar el manejo de memoria.

Métodos y mensajes

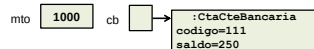
```
class TestUnaCuenta {
    public static void main (String a[]){
        CtaCteBancaria cb = new CtaCteBancaria (111,250);
        cb.depositar(1000);
        System.out.println(cb.obtenerCodigo()+" "+cb.obtenerSaldo());
    }
}
```



El objeto ligado a la variable **cb** recibe el mensaje **depositar** con parámetro real **1000**.

Métodos y mensajes

```
class TestUnaCuenta {
    public static void main (String a[]){
        CtaCteBancaria cb = new CtaCteBancaria (111,250);
        cb.depositar(1000);
        System.out.println(cb.obtenerCodigo()+" "+cb.obtenerSaldo());
    }
}
```



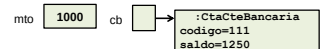
```
public void depositar(float mto){
    /*Incrementa el saldo en mto, Requiere mto > 0*/
    saldo += mto; }

```

El parámetro formal **mto** se inicializa con el valor del parámetro real **m**.

Métodos y mensajes

```
class TestUnaCuenta {
    public static void main (String a[]){
        CtaCteBancaria cb = new CtaCteBancaria (111,250);
        cb.depositar(1000);
        System.out.println(cb.obtenerCodigo()+" "+cb.obtenerSaldo());
    }
}
```



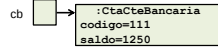
```
public void depositar(float mto){
    /*Incrementa el saldo en mto, Requiere mto > 0*/
    saldo += mto; }

```

Métodos y mensajes

```
class TestUnaCuenta {
    public static void main (String a[]){

        CtaCteBancaria cb = new CtaCteBancaria (111,250);
        cb.depositar(1000);
        System.out.println(cb.obtenerCodigo()+" "+cb.obtenerSaldo());
    }
}
```

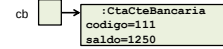


Cuando el método `depositar` termina, `cb` mantiene su valor, pero el estado interno del objeto ligado a la variable `cb` cambió.

Métodos y mensajes

```
class TestUnaCuenta {
    public static void main (String a[]){

        CtaCteBancaria cb = new CtaCteBancaria (111,250);
        cb.depositar(1000);
        System.out.println(cb.obtenerCodigo()+" "+cb.obtenerSaldo());
    }
}
```

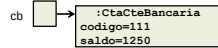


El objeto ligado a la variable `cb` recibe los mensajes `obtenerCodigo()` y `obtenerSaldo()`

Métodos y mensajes

```
class TestUnaCuenta {
    public static void main (String a[]){

        CtaCteBancaria cb = new CtaCteBancaria (111,250);
        cb.depositar(1000);
        System.out.println(cb.obtenerCodigo()+" "+cb.obtenerSaldo());
    }
}
```



111 1250.0

Cuando el control vuelve a `main` se muestra en consola, una cadena de caracteres, la concatenación de los resultados que retornan de `obtenerCodigo()` y `obtenerSaldo()`

Métodos y mensajes

En la POO la comunicación entre objetos se realiza a través de **mensajes**.

Cuando un objeto recibe un mensaje ejecuta un **método**.

La clase del objeto determina el método que se ejecuta en respuesta al mensaje.

En Java la sintaxis de un mensaje es:

```
[< Identificador >].< Identificador > (< Lista de parámetros >)]
```

Métodos y mensajes

CtaCteBancaria	
<<Atributo de clase>> maxDescubierto = 1000	
<<Atributos de instancia>> codigo:entero saldo:real	
<<Constructores>> CtaCteBancaria(c:entero) CtaCteBancaria(c:entero,s:real)	
<<Comandos>> depositar(mto:real) extraer(mto:real):boolean	
<<Consultas>> obtenerCodigo():entero obtenerSaldo():real toString():String mayorSaldo(CtaCteBancaria cta):entero ctaMayorSaldo(CtaCteBancaria cta):CtaCteBancaria	

```
public String toString(){
    return codigo+" "+saldo;
}
```

String es una clase provista por Java.

El nombre `toString()` es estandar para referirse a una consulta que retorna una **cadena de caracteres** cuyo valor es la concatenación de los valores de los atributos del objeto que recibe el mensaje.

El operador `+` permite **convertir** valores de diferentes tipos elementales y **concatenarlos**.

Identidad y estado interno

```
class TestUnDeposito {
    public static void main (String a[]){

        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString());
    }
}
```

En Java el método `main` que inicia la ejecución

Identidad y estado interno

```
class TestUnDeposito {
    public static void main (String a[]) {
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString()); }
}
```

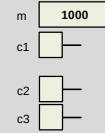


```
01100
10011
10110
01110
01100
10011
10110
10110
10011
1 11
0 0
1
```

Se declaran tres variables de clase **CtaCteBancaria**, las variables no están ligadas

Identidad y estado interno

```
class TestUnDeposito {
    public static void main (String a[]) {
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString()); }
}
```



```
01100
10011
10110
01110
01100
10011
10110
10110
10011
1 11
0 0
1
```

Se declara una variable de tipo elemental y se inicializa con el valor 1000

Identidad y estado interno

```
class TestUnDeposito {
    public static void main (String a[]) {
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString()); }
}
```

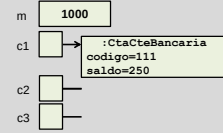


```
01100
10011
10110
01110
01100
10011
10110
10110
10011
1 11
0 0
1
```

main puede acceder a las variables **m**, **c1**, **c2** y **c3**
Estas variables no son visibles para los métodos de la clase **CtaCteBancaria**

Identidad y estado interno

```
class TestUnDeposito {
    public static void main (String a[]) {
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString()); }
}
```



```
01100
10011
10110
01110
01100
10011
10110
10110
10011
1 11
0 0
1
```

- Crea un objeto de software de clase **CtaCteBancaria**
- Liga el objeto a la variable **c1**

Identidad y estado interno

```
class TestUnDeposito {
    public static void main (String a[]) {
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString()); }
}
```

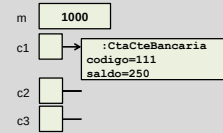


```
01100
10011
10110
01110
01100
10011
10110
10110
10011
1 11
0 0
1
```

El valor de la variable **c1** es una referencia a un objeto de clase **CtaCteBancaria**

Identidad y estado interno

```
class TestUnDeposito {
    public static void main (String a[]) {
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString()); }
}
```

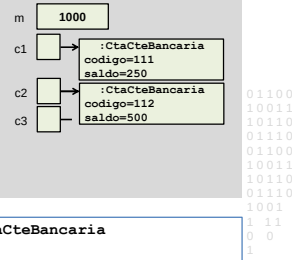


```
01100
10011
10110
01110
01100
10011
10110
10110
10011
1 11
0 0
1
```

El estado interno del objeto ligado a la variable **c1** mantiene los valores 111 para el atributo de instancia **código** y 250 para el atributo de instancia **saldo**.

Identidad y estado interno

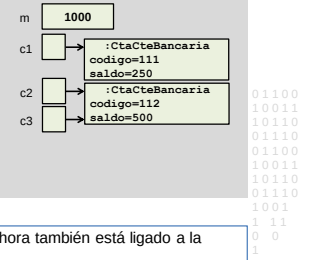
```
class TestUnDeposito {
    public static void main (String a[]) {
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString());
    }
}
```



- Crea un objeto de software de clase **CtaCteBancaria**
- Liga el objeto a la variable **c2**.

Identidad y estado interno

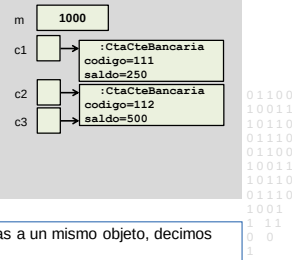
```
class TestUnDeposito {
    public static void main (String a[]) {
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString());
    }
}
```



- Liga el objeto que ya estaba ligado a **c2** ahora también está ligado a la variable **c3**.

Identidad y estado interno

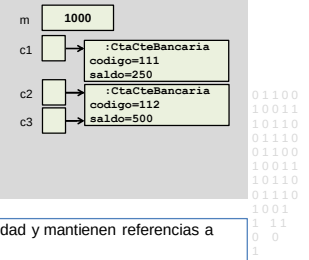
```
class TestUnDeposito {
    public static void main (String a[]) {
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString());
    }
}
```



- Las variables **c2** y **c3** mantienen referencias a un mismo objeto, decimos que **c2** y **c3** tienen la misma **identidad**.

Identidad y estado interno

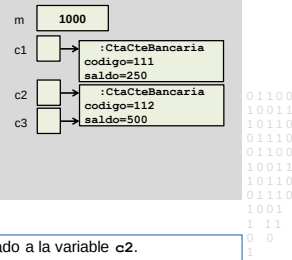
```
class TestUnDeposito {
    public static void main (String a[]) {
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString());
    }
}
```



- Las variables **c1** y **c2** tienen distinta identidad y mantienen referencias a objetos con distinto estado interno.

Identidad y estado interno

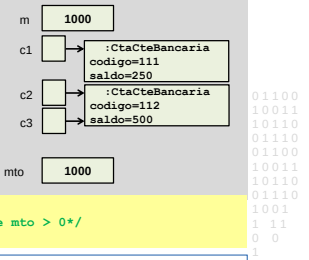
```
class TestUnDeposito {
    public static void main (String a[]) {
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString());
    }
}
```



- Envía el mensaje **depositar** al objeto ligado a la variable **c2**.

Identidad y estado interno

```
class TestUnDeposito {
    public static void main (String a[]) {
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString());
    }
}
```



- El parámetro formal **mto** se inicializa con el valor del parámetro real **m**.

Identidad y estado interno

```
class TestUnDeposito {
    public static void main (String a[]){
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString());
    }
}
```

Diagrama de memoria:

- Variable `m` apunta a un objeto `float` con valor 1000.
- Variable `c1` apunta a un objeto `CtaCteBancaria` con `codigo=111` y `saldo=250`.
- Variable `c2` apunta a un objeto `CtaCteBancaria` con `codigo=112` y `saldo=500`.
- Variable `c3` apunta al mismo objeto `CtaCteBancaria` que `c2` (codigo=112, saldo=500).
- Variable `mt0` (no visible en el código) apunta a un objeto `float` con valor 1000.

El atributo de instancia `saldo` del objeto que recibió el mensaje se incrementa de acuerdo al valor de `mt0`.

Identidad y estado interno

```
class TestUnDeposito {
    public static void main (String a[]){
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString());
    }
}
```

Diagrama de memoria:

- Variable `m` apunta a un objeto `float` con valor 1000.
- Variable `c1` apunta a un objeto `CtaCteBancaria` con `codigo=111` y `saldo=250`.
- Variable `c2` apunta a un objeto `CtaCteBancaria` con `codigo=112` y `saldo=500`.
- Variable `c3` apunta al mismo objeto `CtaCteBancaria` que `c2` (codigo=112, saldo=500).

El método `depositar` termina, la variable `mt0` se destruye y el control vuelve a `main`.

Identidad y estado interno

```
class TestUnDeposito {
    public static void main (String a[]){
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString());
    }
}
```

Diagrama de memoria:

- Variable `m` apunta a un objeto `float` con valor 1000.
- Variable `c1` apunta a un objeto `CtaCteBancaria` con `codigo=111` y `saldo=250`.
- Variable `c2` apunta a un objeto `CtaCteBancaria` con `codigo=112` y `saldo=500`.
- Variable `c3` apunta al mismo objeto `CtaCteBancaria` que `c2` (codigo=112, saldo=500).

Se envía el mensaje `toString` al objeto ligado a la variable `c1`.

Identidad y estado interno

```
class TestUnDeposito {
    public static void main (String a[]){
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString());
    }
}
```

Diagrama de memoria:

- Variable `m` apunta a un objeto `float` con valor 1000.
- Variable `c1` apunta a un objeto `CtaCteBancaria` con `codigo=111` y `saldo=250`.
- Variable `c2` apunta a un objeto `CtaCteBancaria` con `codigo=112` y `saldo=500`.
- Variable `c3` apunta al mismo objeto `CtaCteBancaria` que `c2` (codigo=112, saldo=500).

Retorna una cadena de caracteres concatenando los valores de los atributos `codigo` y `saldo`, separados por un espacio en blanco.

Identidad y estado interno

```
class TestUnDeposito {
    public static void main (String a[]){
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString());
    }
}
```

Diagrama de memoria:

- Variable `m` apunta a un objeto `float` con valor 1000.
- Variable `c1` apunta a un objeto `CtaCteBancaria` con `codigo=111` y `saldo=250`.
- Variable `c2` apunta a un objeto `CtaCteBancaria` con `codigo=112` y `saldo=500`.
- Variable `c3` apunta al mismo objeto `CtaCteBancaria` que `c2` (codigo=112, saldo=500).

El método `toString` de la clase `CtaCteBancaria` termina y el método `main` muestra en consola la cadena de caracteres que retornó.

Identidad y estado interno

```
class TestUnDeposito {
    public static void main (String a[]){
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString());
    }
}
```

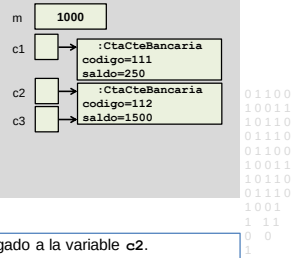
Diagrama de memoria:

- Variable `m` apunta a un objeto `float` con valor 1000.
- Variable `c1` apunta a un objeto `CtaCteBancaria` con `codigo=111` y `saldo=250`.
- Variable `c2` apunta a un objeto `CtaCteBancaria` con `codigo=112` y `saldo=500`.
- Variable `c3` apunta al mismo objeto `CtaCteBancaria` que `c2` (codigo=112, saldo=500).

111 250.0

Identidad y estado interno

```
class TestUnDeposito {
    public static void main (String a[]){
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString());
    }
}
```

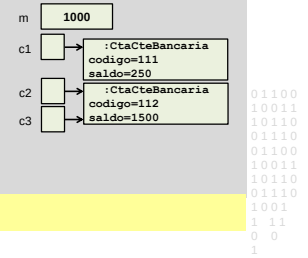


Se envía el mensaje `toString` al objeto ligado a la variable `c2`.

Identidad y estado interno

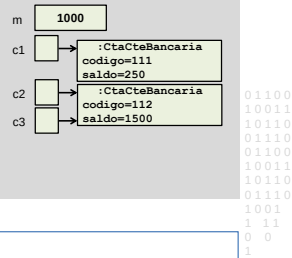
```
class TestUnDeposito {
    public static void main (String a[]){
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString());
    }
}

public String toString(){
    return codigo+" "+saldo;
}
```



Identidad y estado interno

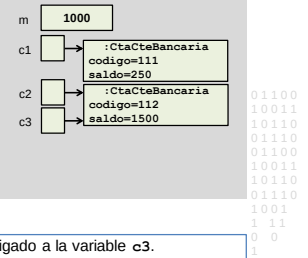
```
class TestUnDeposito {
    public static void main (String a[]){
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString());
    }
}
```



```
111 250.0
112 1500.0
```

Identidad y estado interno

```
class TestUnDeposito {
    public static void main (String a[]){
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString());
    }
}
```

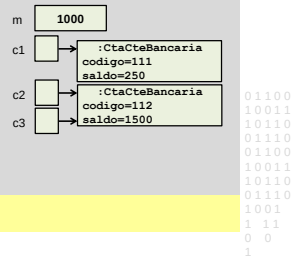


Se envía el mensaje `toString` al objeto ligado a la variable `c3`.

Identidad y estado interno

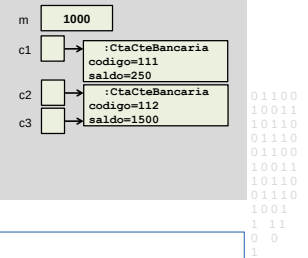
```
class TestUnDeposito {
    public static void main (String a[]){
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString());
    }
}

public String toString(){
    return codigo+" "+saldo;
}
```



Identidad y estado interno

```
class TestUnDeposito {
    public static void main (String a[]){
        CtaCteBancaria c1,c2,c3;
        float m=1000;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        c3 = c2;
        c2.depositar(m);
        System.out.println(c1.toString());
        System.out.println(c2.toString());
        System.out.println(c3.toString());
    }
}
```



```
111 250.0
112 1500.0
112 1500.0
```

Identidad y estado interno

El **estado interno** del objeto almacena los valores de las variables que corresponden a los atributos de instancia del objeto, determinados por su clase.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Parámetros y resultados

CtaCteBancaria
<<Atributo de clase>> maxDescubierto = 1000
<<Atributos de instancia>> codigo:entero saldo:real
<<Constructores>> CtaCteBancaria(c:entero) CtaCteBancaria(c:entero,s:real)
<<Comandos>> depositar(mto:real) extraer(mto:real):boolean
<<Consultas>> obtenerCodigo():entero obtenerSaldo():real toString():String mayorSaldo(CtaCteBancaria cta):entero ctaMayorSaldo(CtaCteBancaria cta): CtaCteBancaria

mayorSaldo(cta: CtaCteBancaria):entero
Requiere cta ligada.
Si el saldo de la cuenta que recibe el mensaje es mayor que el saldo de cta, retorna **el código** de la cuenta que recibe el mensaje, sino retorna el código de cta.

ctaMayorSaldo(cta:CtaCteBancaria):
CtaCteBancaria
Requiere cta ligada.
Si el saldo de la cuenta que recibe el mensaje es mayor que el saldo de cta, retorna **la cuenta** que recibe el mensaje, sino retorna cta.

```
00
11
10
10
00
11
10
10
1
1
```

Parámetros y resultados

```
public int mayorSaldo(CtaCteBancaria cta){
    /*Requiere cta ligada. Si el saldo de la cuenta que recibe el
    mensaje es mayor que el saldo de cta, retorna el código de la
    cuenta que recibe el mensaje, sino retorna el código de cta. */

}

```

```
00
11
10
10
00
11
10
10
1001
1
```

El método **mayorSaldo** recibe como parámetro a una variable que mantiene una referencia a un objeto de clase **CtaCteBancaria**.

Parámetros y resultados

```
public int mayorSaldo(CtaCteBancaria cta){
    /*Requiere cta ligada. Si el saldo de la cuenta que recibe el
    mensaje es mayor que el saldo de cta, retorna el código de la
    cuenta que recibe el mensaje, sino retorna el código de cta. */
    int cod = 0;

    return cod;
}

```

```
00
11
10
10
00
11
10
10
1001
1
```

Se declara una variable local **cod** que va a ser el valor que retorna.
El tipo que retorna es compatible con el tipo que aparece en el encabezamiento del método.

Parámetros y resultados

```
public int mayorSaldo(CtaCteBancaria cta){
    /*Requiere cta ligada. Si el saldo de la cuenta que recibe el
    mensaje es mayor que el saldo de cta, retorna el código de la
    cuenta que recibe el mensaje, sino retorna el código de cta. */
    int cod = 0;
    if (saldo > cta.obtenerSaldo())
        cod = codigo;

    return cod;
}

```

```
00
11
10
10
00
11
10
10
1001
1
```

Si el saldo de la cuenta bancaria que recibe el mensaje es mayor que el saldo de la cuenta corriente que pasó como parámetro, a **cod** se le asigna el código de la cuenta corriente que recibió el mensaje.

Parámetros y resultados

```
public int mayorSaldo(CtaCteBancaria cta){
    /*Requiere cta ligada. Si el saldo de la cuenta que recibe el
    mensaje es mayor que el saldo de cta, retorna el código de la
    cuenta que recibe el mensaje, sino retorna el código de cta. */
    int cod = 0;
    if (saldo > cta.obtenerSaldo())
        cod = codigo;
    else
        cod = cta.obtenerCodigo();
    return cod;
}

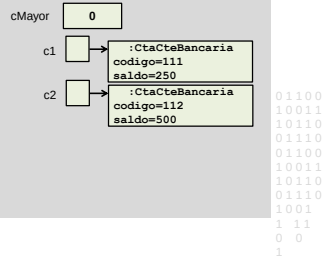
```

```
00
11
10
10
00
11
10
10
1001
1
```

En caso contrario a **cod** se le asigna el código de **cta**.

Parámetros y resultados

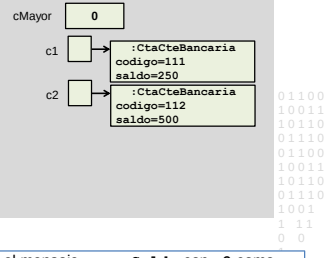
```
class TestSaldo {
public static void main (String a[]) {
    CtaCteBancaria c1,c2;
    int cMayor = 0;
    c1 = new CtaCteBancaria (111,250);
    c2 = new CtaCteBancaria (112,500);
}
```



Parámetros y resultados

```
class TestSaldo {
public static void main (String a[]) {
    CtaCteBancaria c1,c2;
    int cMayor = 0;
    c1 = new CtaCteBancaria (111,250);
    c2 = new CtaCteBancaria (112,500);

    cMayor = c1.mayorSaldo(c2);
}
```



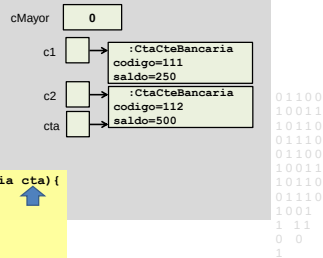
El objeto ligado a la variable **c1** recibe el mensaje **mayorSaldo** con **c2** como parámetro.

Parámetros y resultados

```
class TestSaldo {
public static void main (String a[]) {
    CtaCteBancaria c1,c2;
    int cMayor = 0;
    c1 = new CtaCteBancaria (111,250);
    c2 = new CtaCteBancaria (112,500);

    cMayor = c1.mayorSaldo(c2);

    public int mayorSaldo(CtaCteBancaria cta){
        /*...*/
    }
}
```

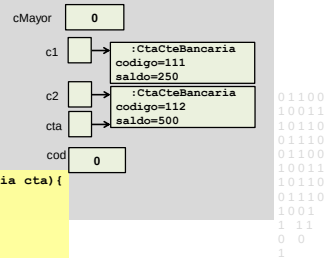


Parámetros y resultados

```
class TestSaldo {
public static void main (String a[]) {
    CtaCteBancaria c1,c2;
    int cMayor = 0;
    c1 = new CtaCteBancaria (111,250);
    c2 = new CtaCteBancaria (112,500);

    cMayor = c1.mayorSaldo(c2);

    public int mayorSaldo(CtaCteBancaria cta){
        /*...*/
        int cod = 0;
    }
}
```

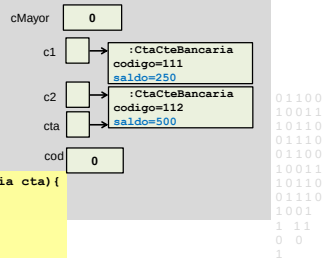


Parámetros y resultados

```
class TestSaldo {
public static void main (String a[]) {
    CtaCteBancaria c1,c2;
    int cMayor = 0;
    c1 = new CtaCteBancaria (111,250);
    c2 = new CtaCteBancaria (112,500);

    cMayor = c1.mayorSaldo(c2);

    public int mayorSaldo(CtaCteBancaria cta){
        /*...*/
        int cod = 0;
        if (saldo > cta.obtenerSaldo())
            cod = codigo;
        else cod = cta.obtenerCodigo();
        return cod; }
}
```

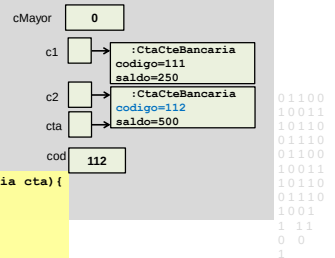


Parámetros y resultados

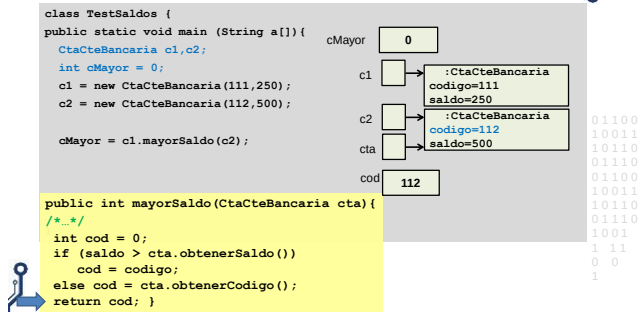
```
class TestSaldo {
public static void main (String a[]) {
    CtaCteBancaria c1,c2;
    int cMayor = 0;
    c1 = new CtaCteBancaria (111,250);
    c2 = new CtaCteBancaria (112,500);

    cMayor = c1.mayorSaldo(c2);

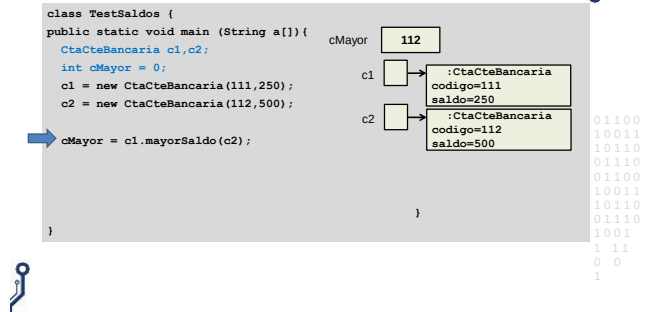
    public int mayorSaldo(CtaCteBancaria cta){
        /*...*/
        int cod = 0;
        if (saldo > cta.obtenerSaldo())
            cod = codigo;
        else cod = cta.obtenerCodigo();
        return cod; }
}
```



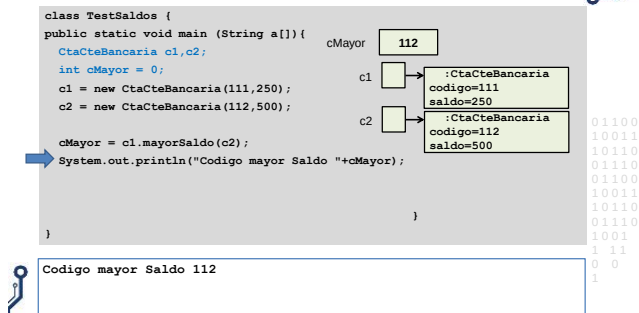
Parámetros y resultados



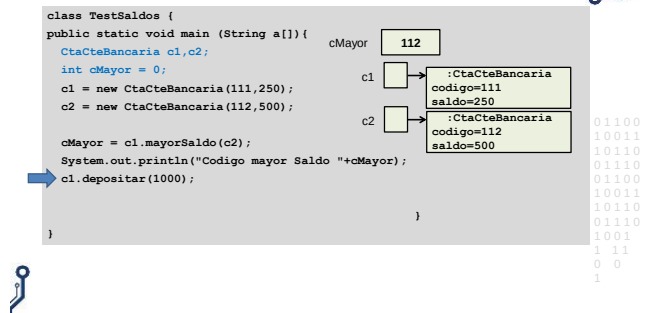
Parámetros y resultados



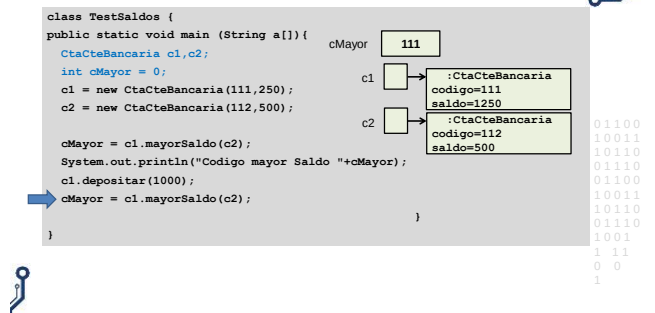
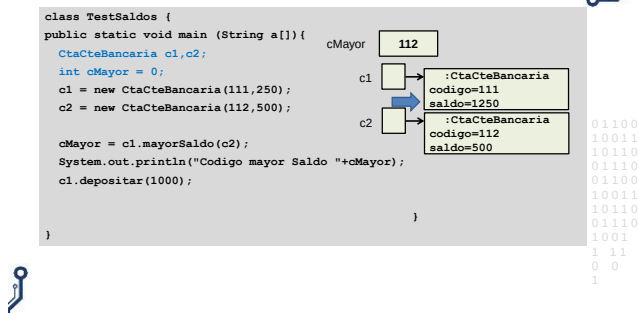
Parámetros y resultados



Parámetros y resultados



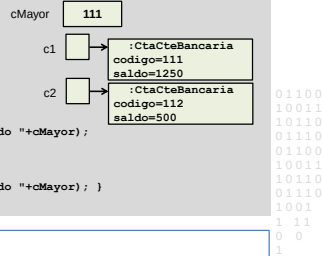
Parámetros y resultados



Parámetros y resultados

```
class TestSaldo {
    public static void main (String a[]) {
        CtaCteBancaria c1, c2;
        int cMayor = 0;
        c1 = new CtaCteBancaria (111, 250);
        c2 = new CtaCteBancaria (112, 500);

        cMayor = c1.mayorSaldo(c2);
        System.out.println("Codigo mayor Saldo "+cMayor);
        c1.depositar(1000);
        cMayor = c1.mayorSaldo(c2);
        System.out.println("Codigo mayor Saldo "+cMayor);
    }
}
```



```
Codigo mayor Saldo 112
Codigo mayor Saldo 111
```

Parámetros y resultados

El pasaje de parámetros en Java es **por valor**.

En el momento que comienza la ejecución de un método se reserva un bloque de memoria para cada **parámetro formal** y se inicializa con el valor del **parámetro real**.

Durante la ejecución, cada parámetro formal **se trata como una variable local**.

Cuando termina la ejecución el bloque de memoria asociado a cada parámetro formal **se libera**.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Parámetros y resultados

CtaCteBancaria
<<Atributo de clase>> maxDescubierto = 1000
<<Atributos de instancia>> codigo: entero saldo: real
<<Constructores>> CtaCteBancaria(c: entero) CtaCteBancaria(c: entero, s: real)
<<Comandos>> depositar(mto: real) extraer(mto: real): boolean
<<Consultas>> obtenerCodigo(): entero obtenerSaldo(): real toString(): String mayorSaldo(CtaCteBancaria cta): entero ctaMayorSaldo(CtaCteBancaria cta): CtaCteBancaria

```
mayorSaldo(cta: CtaCteBancaria): entero
Requiere cta ligada.
Si el saldo de la cuenta que recibe el mensaje es
mayor que el saldo de cta, retorna el código de la
cuenta que recibe el mensaje, sino retorna el
código de cta.

ctaMayorSaldo(cta: CtaCteBancaria):
CtaCteBancaria
Requiere cta ligada.
Si el saldo de la cuenta que recibe el mensaje es
mayor que el saldo de cta, retorna la cuenta que
recibe el mensaje, sino retorna cta.
```

```
00
11
10
10
00
11
10
10
1
1
```

Parámetros y resultados

```
public CtaCteBancaria ctaMayorSaldo(CtaCteBancaria cta) {
    /*Requiere cta ligada. Si el saldo de la cuenta que recibe el
    mensaje es mayor que el saldo de cta, retorna la cuenta que
    recibe el mensaje, sino retorna cta. */

}
```

El método **ctaMayorSaldo** retorna como resultado una referencia a un objeto de clase **CtaCteBancaria**.

```
00
11
10
10
00
11
10
10
1001
1
```

Parámetros y resultados

```
public CtaCteBancaria ctaMayorSaldo(CtaCteBancaria cta) {
    /*Requiere cta ligada. Si el saldo de la cuenta que recibe el
    mensaje es mayor que el saldo de cta, retorna la cuenta que
    recibe el mensaje, sino retorna cta. */
    CtaCteBancaria may = null;

}
```

```
00
11
10
10
00
11
10
10
1001
1 11
1
```

Se declara una variable local llamada **may** de clase **CtaCteBancaria**

Parámetros y resultados

```
public CtaCteBancaria ctaMayorSaldo(CtaCteBancaria cta) {
    /*Requiere cta ligada. Si el saldo de la cuenta que recibe el
    mensaje es mayor que el saldo de cta, retorna la cuenta que
    recibe el mensaje, sino retorna cta. */
    CtaCteBancaria may = null;

}
```

```
00
11
10
10
00
11
10
10
1001
1
```

Las variables **may** y **cta** clase **CtaCteBancaria** solo son visibles en el método **ctaMayorSaldo**

Parámetros y resultados

```
public CtaCteBancaria ctaMayorSaldo(CtaCteBancaria cta){
    /*Requiere cta ligada. Si el saldo de la cuenta que recibe el
    mensaje es mayor que el saldo de cta, retorna la cuenta que
    recibe el mensaje, sino retorna cta. */
    CtaCteBancaria may = null;

    return may;
}
```

Retorna como **resultado** el valor de la variable local **may** de clase **CtaCteBancaria**

Parámetros y resultados

```
public CtaCteBancaria ctaMayorSaldo(CtaCteBancaria cta){
    /*Requiere cta ligada. Si el saldo de la cuenta que recibe el
    mensaje es mayor que el saldo de cta, retorna la cuenta que
    recibe el mensaje, sino retorna cta. */
    CtaCteBancaria may = null;
    if (saldo > cta.obtenerSaldo())
        may = this;

    return may;
}
```

Si el saldo de la cuenta que recibe el mensaje es mayor que el saldo de la cuenta corriente que pasa como parámetro, retorna la referencia al objeto que recibe el mensaje.

Parámetros y resultados

```
public CtaCteBancaria ctaMayorSaldo(CtaCteBancaria cta){
    /*Requiere cta ligada. Si el saldo de la cuenta que recibe el
    mensaje es mayor que el saldo de cta, retorna la cuenta que
    recibe el mensaje, sino retorna cta. */
    CtaCteBancaria may = null;
    if (saldo > cta.obtenerSaldo())
        may = this;

    return may;
}
```

En Java la variable **this** mantiene la referencia al objeto que recibe el mensaje

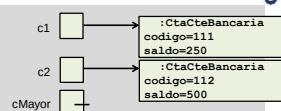
Parámetros y resultados

```
public CtaCteBancaria ctaMayorSaldo(CtaCteBancaria cta){
    /*Requiere cta ligada. Si el saldo de la cuenta que recibe el
    mensaje es mayor que el saldo de cta, retorna la cuenta que
    recibe el mensaje, sino retorna cta. */
    CtaCteBancaria may = null;
    if (saldo > cta.obtenerSaldo())
        may = this;
    else
        may = cta;
    return may;
}
```

En caso contrario a la variable **may** se le asigna el parámetro formal, es decir la referencia al objeto que pasó como parámetro.

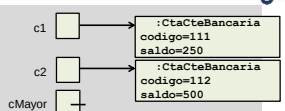
Parámetros y resultados

```
class TestSaldo {
    public static void main (String a[]){
        CtaCteBancaria c1,c2;
        CtaCteBancaria cMayor = null;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
    }
}
```



Parámetros y resultados

```
class TestSaldo {
    public static void main (String a[]){
        CtaCteBancaria c1,c2;
        CtaCteBancaria cMayor = null;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        cMayor = c1.ctaMayorSaldo(c2);
        System.out.println(cMayor.toString());
    }
}
```



Envía el mensaje **ctaMayorSaldo** al objeto ligado a la variable **c1** con el objeto ligado a **c2** como parámetro .

Parámetros y resultados

```
class TestSaldo {
    public static void main (String a[]) {
        CtaCteBancaria c1,c2;
        CtaCteBancaria cMayor = null;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        cMayor = c1.ctaMayorSaldo(c2);
        System.out.println(cMayor.toString());
    }
}

public CtaCteBancaria ctaMayorSaldo(CtaCteBancaria cta){
    /*...*/
    CtaCteBancaria may = null;
    if (saldo > cta.obtenerSaldo())
        may = this;
    else    may = cta;
    return may;
}
```

Parámetros y resultados

```
class TestSaldo {
    public static void main (String a[]) {
        CtaCteBancaria c1,c2;
        CtaCteBancaria cMayor = null;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        cMayor = c1.ctaMayorSaldo(c2);
        System.out.println(cMayor.toString());
    }
}

public CtaCteBancaria ctaMayorSaldo(CtaCteBancaria cta){
    /*...*/
    CtaCteBancaria may = null;
    if (saldo > cta.obtenerSaldo())
        may = this;
    else    may = cta;
    return may;
}
```

Parámetros y resultados

```
class TestSaldo {
    public static void main (String a[]) {
        CtaCteBancaria c1,c2;
        CtaCteBancaria cMayor = null;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        cMayor = c1.ctaMayorSaldo(c2);
        System.out.println(cMayor.toString());
    }
}

public CtaCteBancaria ctaMayorSaldo(CtaCteBancaria cta){
    /*...*/
    CtaCteBancaria may = null;
    if (saldo > cta.obtenerSaldo())
        may = this;
    else    may = cta;
    return may;
}
```

Parámetros y resultados

```
class TestSaldo {
    public static void main (String a[]) {
        CtaCteBancaria c1,c2;
        CtaCteBancaria cMayor = null;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        cMayor = c1.ctaMayorSaldo(c2);
        System.out.println(cMayor.toString());
    }
}

public CtaCteBancaria ctaMayorSaldo(CtaCteBancaria cta){
    /*...*/
    CtaCteBancaria may = null;
    if (saldo > cta.obtenerSaldo())
        may = this;
    else    may = cta;
    return may;
}
```

Parámetros y resultados

```
class TestSaldo {
    public static void main (String a[]) {
        CtaCteBancaria c1,c2;
        CtaCteBancaria cMayor = null;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        cMayor = c1.ctaMayorSaldo(c2);
        System.out.println(cMayor.toString());
    }
}
```

Cuando ctaMayorSaldo termina retorna como resultado una referencia que se asigna a la variable cMayor, que ahora es de clase CtaCteBancaria.

Parámetros y resultados

```
class TestSaldo {
    public static void main (String a[]) {
        CtaCteBancaria c1,c2;
        CtaCteBancaria cMayor = null;
        c1 = new CtaCteBancaria (111,250);
        c2 = new CtaCteBancaria (112,500);
        cMayor = c1.ctaMayorSaldo(c2);
        System.out.println(cMayor.toString());
    }
}
```

Envía el mensaje toString al objeto ligado a la variable cMayor de clase CtaCteBancaria, en consola se muestra el estado interno.

La clase tester

La **clase tester** verifica que la clase cumple con sus **responsabilidades** y los servicios se comportan de acuerdo a la **funcionalidad** y las **restricciones** especificadas, para un conjunto de **casos de prueba**.

Los casos de prueba pueden ser:

- Fijos.
- Leídos de un archivo.
- Ingresados por el usuario por consola o a través de una interfaz gráfica.
- Generados al azar.

```
1100
0011
0110
1110
1100
0011
0110
1110
001
11
0
1
```

Parámetros y resultados

```
class TestSaldo {
public static void main (String a[]){
    CtaCteBancaria c1,c2;
    CtaCteBancaria cMayor = null;
    c1 = new CtaCteBancaria (111,250);
    c2 = new CtaCteBancaria (112,500);

    cMayor = c1.ctaMayorSaldo(c2);
}
}
```

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
111
00
1
```

Dibuje el diagrama de objetos al completarse la ejecución del tester

Cambio de diseño

CajaAhorro
<<Atributo de clase>> maxDescubierto = 1000
<<Atributos de instancia>> codigo:entero saldo:real
<<Constructores>> CajaAhorro(c:entero) CajaAhorro(c:entero,s:real)
<<Comandos>> depositar(mto:real) extraer(mto:real)
<<Consultas>> obtenerCodigo():entero obtenerSaldo():real puedeExtraer(mto:real):boolean toString():String mayorSaldo(CtaCteBancaria cta):entero ctaMayorSaldo(CtaCteBancaria cta): CtaCteBancaria

puedeExtraer(mto:real):boolean
Si 0 > mto > saldo+maxDescubierto retorna false,
sino retorna true

extraer(mto:real)
Requiere 0 <= mto <= saldo+maxDescubierto.
Decrementa el saldo en mto

Implemente la clase CajaAhorro para este diseño y una nueva clase TesterExtraer para verificar los servicios puedeExtraer y extraer con valores leídos por consola.

```
00
11
00
00
00
01
0110
```

En esta clase

- Clases proveedoras y clientes
- Variables, objetos y referencias
- Diagrama de objetos
- Métodos y mensajes
- Identidad y estado interno
- Parámetros y resultados
- Ejemplo: Cajero automático

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
111
00
1
```